JKSCI

Journal of The Korea Society of Computer and Information

Design and PoC Validation of a Secure On-Premises Meeting Intelligence Pipeline for Construction Enterprises

Jaehwan Kim[1] · Jaeseong Yun[2] · Jongseok Lee[3] · Kyungmin Kim[4]

[1]aSSIST, Seoul, Republic of Korea; [2]XAIKOREA Inc., Seongnam, Republic of Korea; [3]RAD CNS Co., Ltd., Republic of Korea; [4]Security Platform, Republic of Korea

First and Corresponding Author: Jaehwan Kim (kjh861213@naver.com)

[Abstract]

This study evaluates artifact validation and checkpoint recovery in an on-premises meeting intelligence pipeline for construction enterprises. With the ASR model and decoding configuration held constant, we compared full restart, syntax-based reuse, and reuse with combined integrity and execution-condition checks. We processed the first 120 seconds of 16 AMI test recordings in independent 30-second chunks. All 912 runs terminated: 48 no-fault controls and 864 fault-injection runs. Full restart and combined validation preserved the uninterrupted reference transcript in every fault-injection run, whereas syntax-based reuse produced incorrect transcripts in 96 text-corruption runs. Following stop-only interruption after the third saved chunk, the mean of the recording-level median recovery latencies was 26.19 seconds for both reuse policies and 83.52 seconds for full restart. The recording-level paired reduction for combined validation was approximately 57.3 seconds. These findings demonstrate the roles of content validation and valid-chunk reuse under the tested conditions. They do not establish whole-system security or recognition accuracy for Korean construction meetings. Establishing the incremental benefit of execution-condition checks requires comparison with a checksum-only policy.

▸ Keywords: On-Premises AI, Meeting Intelligence, Artifact Integrity, Checkpoint Recovery, Fault Injection

I Introduction

Organizational speech transcription systems must support recovery from interruptions to long-running jobs while maintaining recognition quality. Reusing transcripts from previously processed chunks can reduce computation. However, accepting partial files, readable files with altered content, or files generated from different inputs or settings as valid checkpoints can allow a job to complete without preserving its transcript. Execution completion and transcript preservation must therefore be evaluated separately.

Whisper and WhisperX provide foundations for speech recognition and temporal alignment [1,2]. Recovery policies determine which artifacts produced before an interruption can be trusted and which chunks require recomputation. We investigate whether recovery can preserve the transcript of an uninterrupted reference run using the same model while reducing recomputation after specified faults. Because this process may also preserve recognition errors in the original output, we distinguish transcript preservation during recovery from recognition accuracy against a human reference transcript.

Our earlier proof of concept (PoC) for construction meeting processing integrated speech recognition, speaker diarization, summarization, storage, and key management. Its design principles addressed data and key boundaries, artifact completeness, checkpoint recovery, and distinctions between levels of evidence. This study translates the artifact-completeness and recovery principles into explicit rules for transcript-chunk reuse and evaluates the associated recovery costs. It examines measurable recovery behavior within the existing architecture; it neither introduces a new speech model nor establishes the security of the entire pipeline. The experiments use public English meeting recordings, and applicability to Korean construction meetings requires further validation.

We address two research questions (RQs). RQ1 examines how transcript preservation varies across reuse policies under specified file-corruption conditions. RQ2 examines how recomputation and latency vary with fault type and interruption position among runs that preserve the transcript. Both questions are investigated using the completed E1 experiment. The incremental benefit of adding execution-condition checks to hash checks, and comparisons with existing workflows, require separate validation.

This empirical systems study compares recovery policies for speech transcription performed in independent chunks. With audio, model, and fault conditions held constant, we evaluate transcript preservation and computational cost jointly. The results characterize the consequences of basing reuse decisions solely on file readability and the cost savings associated with retaining valid chunks. Input, model, and code identifiers, together with execution records that retain failures, make the comparison traceable.

We interpret the scope of validation separately from its computational cost. Comparing an output with its stored hash can detect changes to the output bytes, whereas determining whether a valid artifact from another job is suitable for the current input and settings requires an additional check. Because E1 combines hash and execution-condition checks, its observed improvements are attributed to the combined policy. Isolating the incremental benefit of each check and comparing it with existing implementations remain tasks for subsequent validation.

II Related Work and Study Scope

1 Speech Processing and Recovery Preservation

Whisper is an automatic speech recognition (ASR) model trained on large-scale weakly supervised

audio-transcript data [1]. WhisperX adds alignment to locate words in time [2], while pyannote.audio supports speaker diarization by grouping speech segments by speaker [3]. These tasks address what was said, when it was said, and who said it. Our focus is whether a generated transcript remains unchanged after interruption and restart. We therefore hold the recognition model and chunk boundaries constant and vary only the recovery policy.

2 Checkpoint Recovery and Fault Injection

Checkpointing allows computation to resume from saved intermediate state rather than restart from the beginning. In their survey of checkpoint- and log-based recovery, Elnozahy et al. identify recovery-state consistency and the amount of repeated computation as central concerns [4]. In our implementation, each completed audio-chunk transcript serves as an application-level checkpoint. If three of four chunks remain valid, only the final chunk requires processing. A retained output must nevertheless be recomputed if it is corrupt or was generated under conditions incompatible with the current input. Suitability for reuse therefore matters as much as the presence of stored output.

Fault injection deliberately introduces interruptions or corruption to assess detection and recovery behavior. Hsueh et al. describe an experimental approach that coordinates injection targets, mechanisms, and observation methods [5]. We apply three fault types to the same audio rather than wait for naturally occurring failures. Stop-only interruption tests reuse of valid results; JSON truncation tests the handling of unreadable files; and content modification tests validation of readable but altered content. Holding other conditions constant allows policy outcomes to be related to the checks each policy performs. Equal run counts across conditions support controlled comparison but do not represent operational fault frequencies.

3 Execution-Condition-Based Caching and Crash Consistency

Caching retains computed results for later reuse. Its central requirement is that a stored result correspond to the task currently requested. Bazel's remote cache distinguishes result metadata associated with inputs and an execution specification from the storage of output content [6]. An unchanged transcript file, for example, is not necessarily suitable for reuse if the input audio has changed. We apply this established distinction to transcript chunks by checking both content integrity and compatibility with the current execution conditions.

Nextflow, introduced by Di Tommaso et al., supports computational reproducibility by managing execution environments and workflows [7]. Cache reuse depends on task hashes incorporating inputs, scripts, containers, and related information, together with stored outputs [8]. DVC similarly manages stage re-execution and output restoration using declared dependencies and parameters [9]. The experimental data-lineage feature introduced in Nextflow 25.04 records relationships between output checksums and the tasks that produced them [10]. Snakemake's between-workflow cache also identifies results using inputs, parameters, and the software environment [11]. Provenance recording, reuse decisions, and output-content validation serve different roles depending on implementation and configuration; feature descriptions and actual fault responses must therefore be compared separately.

Crash-consistency research examines how requested writes relate to the state recoverable after a crash. Pillai et al. show that application update procedures depend on filesystem persistence properties and that these dependencies affect crash consistency [12]. An interruption between updates to a transcript file and its validation record can therefore leave the two in inconsistent states. E1 injects interruption after acknowledgment of a

completed chunk save, whereas a separate file experiment terminates the writing process between individual storage steps. The two experiments address reuse decisions and storage ordering at distinct levels.

4 Scope of On-Premises Deployment

On-premises deployment allows an organization to manage models and data within its own environment. NIST's zero-trust guidance emphasizes verifying trust in subjects and resources independently of physical or network location [13]. Our recovery evaluation assumes trusted inputs, models, execution code, and controller. Authentication, encryption, key separation, administrative privileges, and log management remain separate security requirements for an operational system.

These lines of research address the conditions under which stored results can be reused and the computation that must be repeated after interruption. Checkpointing concerns recovery-state consistency, workflow systems connect inputs and settings to outputs, and crash-consistency studies examine states left by interrupted storage operations. We bring these perspectives to speech transcription by measuring transcript preservation and recovery cost with the input and model held constant. The present contribution is a controlled comparison under specified faults, supported by reproducible evaluation records. Table 1 summarizes the further comparisons needed to isolate execution-condition checks and assess differences from existing tools.

Table 1. Scope of comparison with related work

Study or framework	Main focus	Relation to this study
Whisper and WhisperX [1,2]	Recognition and alignment	Whisper is E1's ASR foundation
pyannote.audio [3]	Speaker diarization	Diarization component of the earlier application
Bazel remote cache [6]	Task- and content-based reuse	Prior distinction between task compatibility and output integrity
Nextflow · DVC · Snakemake [8,9,10,11]	Reuse based on inputs, settings, and environment	Direct comparison pending; equivalent configurations required
Application crash consistency [12]	Storage updates and persistence	Distinguishes process-termination evidence from power-loss evidence
This study	Transcript preservation and recomputation cost	Policy comparison under specified faults using the same worker; failures retained

III Selective Recovery with Execution-Condition Validation

1 Tasks and Trust Boundaries

An artifact is a file containing the transcript and associated metadata generated for an audio chunk. Each recording is divided into four chunks. Artifact A_i for chunk i contains a transcript string (text), a list of segment timing and related information (segments), and the execution conditions under which it was generated (binding). Artifacts are stored in JavaScript Object Notation (JSON). A separate validation record L_i stores the file's SHA-256 (Secure Hash Algorithm 256-bit) hash and execution conditions. This hash detects changes to the file bytes; it does not authenticate the author of the validation record as a digital signature would.

Execution conditions B_i comprise the chunk identifier, the hash of the input audio bytes, the hash of the model weights, and the hash of the

protocol file containing the inference settings. The controller derives the expected conditions B_i* for the current task from the frozen input specification. It compares these values with the conditions stored in the artifact and validation record to identify outputs generated from different inputs, models, or settings. Execution code and the container image are identified in a separate frozen manifest. The information used to decide reuse in E1 is thus related to, but distinct from, the information needed to reproduce the experimental environment.

We assume that the controller, input specification, model, execution code, and validation records remain unchanged from their specified states. E1 alters output files while preserving their validation records to test whether the recorded validation information detects those changes.

Fig. 1 illustrates how audio chunks and execution conditions determine the artifacts available for reuse during recovery. Its three-dimensional layout distinguishes the logical components.

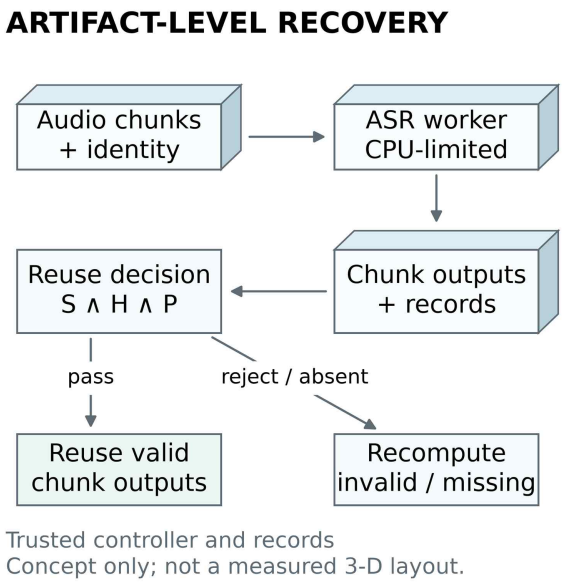


Fig. 1. Conceptual three-dimensional view of transcript-artifact generation, validation, and reuse.

2 Reuse Decisions

Reuse validation addresses three questions. First, can the file be parsed and its required fields interpreted? The syntax condition S(A_i) is a minimal format check: it requires parseable JSON, a string-valued text field, and a list-valued segments field. An empty transcript is permitted for a silent chunk. Second, have the file bytes changed since storage? The integrity condition H(A_i,L_i) compares their current hash with the value in the validation record. Third, was the artifact generated for the task now being performed? The execution-condition check P(A_i,L_i,B_i*) compares the conditions recorded in the artifact and validation record with those required by the current task.

Suppose that the current task requires model B, but a transcript generated by model A and its matching validation record remain available. If the JSON is parseable and the file hash matches the record, checks S and H pass. Check P identifies the mismatch between the model that generated the transcript and the model required for the current task. This example illustrates the distinct roles of the checks; it is not E1's content-modification condition. The combined policy reuses a chunk only if all three checks pass and reruns inference otherwise.

The reuse rule is $R_i = S(A_i) \wedge H(A_i,L_i) \wedge P(A_i,L_i,B_i*)$, where A_i is the transcript file, L_i is its validation record at storage time, B_i* specifies the execution conditions required for the current task, and $\wedge$ requires every condition to hold. An artifact passing all three checks is reused. Check S rejects truncated JSON, and check H rejects altered transcript bytes, causing the affected chunks to be recomputed. An artifact generated by another model can pass S and H when accompanied by its matching record, but P detects the mismatch with the required model. These examples explain the decision rule; the incremental effect of P requires evaluation beyond E1.

For each chunk, the worker reads the expected execution conditions and applies the checks required by the selected policy. It reuses eligible artifacts, runs inference on the remaining chunks,

and saves their outputs and validation records. The job terminates once all chunks are available. The evaluator then compares the resulting transcript with the uninterrupted reference. Execution status and transcript-preservation outcomes are recorded separately.

Full restart reruns inference for every chunk, whereas the syntax-based policy reuses outputs satisfying S. E1's combined-validation policy applies S, H, and P. The output-hash-only policy in the supplementary experiment applies S and H and is designed to assess the incremental effect of P through comparison with combined validation.

Fig. 2 maps the questions asked by the three checks to the policies compared experimentally. File readability, preservation of stored content, and suitability for the current execution are distinct properties.

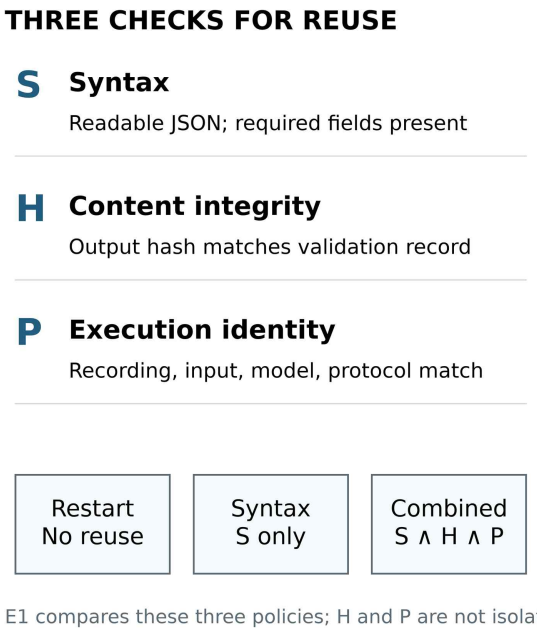


Fig. 2. Roles of syntax (S), content-hash (H), and execution-condition (P) checks and the policies compared in E1.

Syntax-based reuse serves as a minimal-validation comparator, while full restart provides a cost baseline that recomputes every chunk. An output-hash check is sufficient to detect E1's content-modification fault. Isolating the incremental benefit of execution-condition check P therefore requires an output-hash-only comparator.

Here, a result is committed when a recovery job writes its result files and terminates. If the committed result differs from the reference transcript, the run is classified as a correctness failure and recorded as incorrect_commit in the raw data.

3 Storage and Result Commit

Interpreting the state left by an interrupted write requires distinguishing the transcript-file update from the validation-record update. The E1 worker writes the transcript to a temporary file and replaces the output file, then updates the validation record in the same manner. Replacing each file separately does not commit the pair as a single operation. An interruption between these updates can leave the artifact and its record in inconsistent states. E1 terminates the worker after acknowledgment of completed chunk processing; the separate file experiment examines interruptions at the intervening storage boundaries.

After restart, the worker validates each chunk, reuses eligible outputs, and runs inference for the remainder. Once all chunks are available, it records the termination status and compares the list of chunk-level transcript strings with the uninterrupted reference. Exact equality of these lists is E1's correctness criterion. The supplementary experiment additionally distinguishes transcript-string equality from equality of the complete structured output.

4 Interpretation of Decisions and Dependency Scope

The evaluation distinguishes reuse eligibility, preservation of the reference transcript, and recognition quality against a human reference. Reuse eligibility concerns whether an artifact is suitable for the current task; preservation concerns whether the recovered output equals the uninterrupted output; and recognition quality concerns whether that output accurately represents

the speech. Reproducing a sentence originally misrecognized by the model preserves the transcript but also preserves its recognition error. Timing metadata may likewise differ even when the text is identical. E1 therefore assesses exact equality of chunk-level strings, while the supplementary experiment separately records structured-output equality and execution-condition compliance.

The current implementation decodes each 30-second chunk independently, without using transcripts from other chunks as input. Reprocessing one chunk therefore does not require recomputation of its neighbors. In long-form transcription conditioned on preceding text, however, changing an earlier chunk may affect subsequent decoding. Such a system must record both the context and the generation conditions and determine whether recomputation needs to propagate to later chunks. Our chunk-level recovery findings apply to the independent-processing setting.

5 Operational Controls

Experiments run one job at a time in an isolated container with read-only access to the model, code, and inputs. Central processing unit (CPU) usage is capped at 2 CPUs, and both the memory limit and the combined memory-plus-swap limit are set to 8 GiB. Graphics processing unit (GPU) access and external networking are disabled. The controller monitors existing service states and available memory. If a previously healthy service deteriorates or resource criteria are violated, it stops only the experimental job. This monitoring assesses service execution state.

IV Experimental Design and Analysis

1 Data and Fixed Conditions

The Augmented Multi-party Interaction (AMI) corpus contains public multi-party meeting recordings [14]. We selected 16 test recordings from a split manifest pinned to a fixed revision [15]. For each recording, we retained the first 120.000 seconds, including silence, and divided them into four consecutive 30-second chunks. Inputs were 16 kHz, single-channel, 16-bit pulse-code modulation (PCM) audio. The original waveforms and fixed intervals were preserved without alteration. The total duration of unique input audio was 1,920 seconds.

The 16 recordings belong to four series: EN2002, ES2004, IS1009, and TS3003. Repeated runs of a recording are technical replicates used to characterize processing-time variation. As Hurlbert's discussion of pseudoreplication emphasizes, the number of measurements must be distinguished from the number of independent samples [16]. We report descriptive summaries that acknowledge dependence within recordings and series. We also disclose that test-data results were inspected before the follow-up experiments were designed.

Inference used Whisper large-v3-turbo on a CPU with two threads. The language was fixed to English, temperature to 0, and beam size to 1; fp16 and conditioning on preceding chunk text were disabled. SHA-256 identifiers for the model weights, execution image, protocol, input manifest, and schedule were frozen. These settings and identifiers specify the conditions required to reproduce the experiment.

The E1 worker calls load_model and transcribe directly in the Python implementation of OpenAI Whisper. It implements only the ASR stage and uses AMI Mix-Headset as the fixed input channel. The shared aarch64 host has 10 Cortex-X925 cores and 10 Cortex-A725 cores, but the experimental container was limited to the equivalent of 2 CPUs. The frozen container image identifies the execution environment; hashes of the model, execution code, and input manifest are retained alongside it. Appendix C describes these identifiers.

2 Fault Matrix and Execution Order

The main experiment, E1, crosses three policies, three fault types, two interruption positions, and

three technical replicates. It contains 16×3×3×2×3=864 fault-injection runs and 16×3=48 no-fault controls, yielding 912 runs in total. For each recording, an uninterrupted run over all chunks supplies the reference transcript, against which the no-fault controls were checked. The remaining runs followed a frozen schedule generated with seed 20261026.

After the worker saves the first or third chunk and sends a readiness acknowledgment, the controller terminates it with SIGKILL. In the stop-only condition, recovery begins without further changes to the files. JSON truncation removes half of the first chunk's JSON bytes. Content modification replaces that chunk's text with CORRUPTED OUTPUT while retaining valid JSON structure. Both corruption conditions leave the validation record unchanged. The intended SIGKILL is part of the fault-injection protocol and is distinguished from unexpected execution failures, including abnormal worker termination or monitoring failure.

The procedure begins with uninterrupted processing of all four chunks to generate the reference transcript. Each fault-injection run then performs actual inference from the beginning through the first or third chunk and waits for acknowledgment that the output files and validation records have been saved. The controller terminates the worker, applies the specified corruption, and starts a new worker using the same policy. This worker evaluates each chunk for reuse and runs inference only where reuse is rejected. Finally, the four recovered transcript strings are compared with the reference. Pre-interruption inference, intended interruption, corruption, recovery, and evaluation are recorded as distinct stages.

Table 2. Design of the main E1 experiment

Item	Specification
Input	First 120 seconds of each of 16 recordings; 4 series
Processing unit	Four 30-second chunks; no cross-chunk transcript conditioning
Policies	Full restart; syntax checks; combined validation (S+H+P)
Fault	Stop only; JSON truncation; text modification preserving syntax
Interruption position	After saving chunk 1 or 3
Replicates and runs	3 technical replicates per fault condition; 864 fault runs; 48 no-fault controls
Correctness	Exact chunk-string equality with the same recording's clean full-restart transcript
Limits	600-second limit per job; unexpected failures and partial results retained

Fig. 3 summarizes E1's fault-injection sequence. Post-interruption modification targets the first chunk's artifact, and the recovered transcript is compared with the uninterrupted reference.

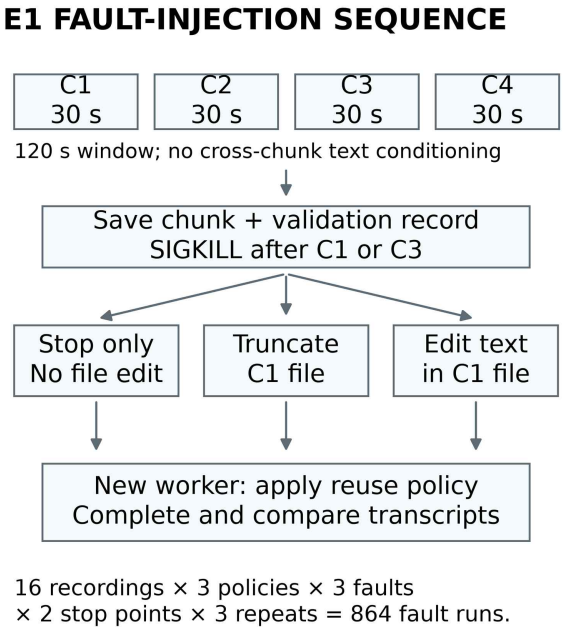


Fig. 3. E1 sequence of storage, interruption, modification, and recovery. C1–C4 are 30-second audio chunks; interruption occurs at C1 or C3 after the save acknowledgment.

The no-fault controls comprise 48 runs: one for each combination of 16 recordings and 3 policies. Fault injection comprises 16 recordings × 3 faults × 2 interruption positions × 3 policies × 3 replicates = 864 runs. The denominator for fault-recovery outcomes is 864.

3 Outcome Measures and Conditional Timing Analysis

Recovery correctness is assessed against the transcript produced from the same input and model without a fault. Comparing the recovered list of chunk-level strings with this reference identifies changes introduced during recovery. A run that terminates normally but returns a different list is classified as an incorrect commit. Execution failures, timeouts, and no-fault controls are recorded separately.

Recomputed audio duration C is the sum of the durations of chunks inferred after the fault. Post-fault latency L runs from immediately before SIGKILL to immediately after the recovery worker terminates and both its result and the reference transcript have been read. It includes corruption, worker restart, model loading, validation, inference, and storage. Total elapsed time T starts immediately before the first worker launch and ends at the same point, thereby including pre-interruption inference. Writing the final result.json, preparing the container, and waiting between batches are separate operational costs. Because L includes experimental control and result reading, relating it to user-perceived recovery time requires consistent measurement boundaries. L is the primary timing measure; Table 5 and Appendix E provide a post hoc analysis of the recorded T values.

To account for differences in speech content and processing difficulty, we compare policies within the same recording. For policy a relative to full restart, the differences are ΔC=C_a-C_restart and Δ L=L_a-L_restart; negative values indicate lower cost. We first calculate paired differences for the same recording, fault, interruption position, and replicate, then take the median of the three differences for each recording. We finally average these recording-level medians with equal weights. Taking the median limits the influence of a single large timing fluctuation. A recording is eligible only if both policies preserve its reference transcript in all three replicates; every condition in Table 4 includes 16 recordings. This order of aggregation can produce a different estimate from the difference between separately calculated policy medians.

Recovery performance encompasses both correctness and speed. Averaging latencies from runs that quickly return incorrect transcripts can give a misleading picture of performance. We therefore retain all correctness failures in Table 3 and condition timing comparisons on correct recovery, explicitly identifying excluded conditions and denominators. Reusing k chunks of 30 seconds saves 30k seconds of recomputed audio. Elapsed time also depends on model loading, validation, storage, and host state. Reporting audio duration and latency separately reveals how reduced computation translates into time savings.

4 Role of Supplementary Validation

After examining intermediate E1 results, we designed a supplementary experiment to isolate the contribution of execution-condition check P. It adds an output-hash-only comparator and conditions involving stale but valid artifact-record pairs. Supplementary designs and results are maintained under separate protocols and paths, preserving all original frozen results and failures.

Supplementary validation is organized by the type of evidence it provides. Storage-boundary experiment E3a uses 14 actual process terminations and 2 uninterrupted controls to examine policy decisions for the files and validation records left at each interruption point. Development integration validation uses donor artifacts generated with a different model or inference setting to test two ASR jobs from interruption through recovery. These experiments assess file-selection rules and

implementation feasibility and cost, respectively, and are reported separately from E1. A supplementary main experiment has been frozen at 960 runs, comprising 64 no-fault and 896 fault-injection runs, with 32 additional donor-generation runs, and is in progress. Policy effects will be evaluated separately when its complete results are available.

V Results

1 Execution Completion and Transcript Preservation

All 912 runs terminated normally, with no unexpected execution failures or timeouts. The 816 runs matching the reference transcript comprise 48 no-fault controls and 768 fault-injection runs. The remaining 96 of the 864 fault-injection runs committed incorrect transcripts.

Each fault-policy cell in Table 3 contains 96 runs: 16 recordings, two interruption positions, and three replicates. Under content modification, syntax-based reuse committed an incorrect transcript in all 96 runs. The injected change altered only text while preserving JSON syntax and field types, so the policy accepted the modified artifact. These correctness failures occurred despite normal execution termination. Under stop-only interruption and truncation, all three policies preserved the transcript in all 96 runs per cell. Combined validation and full restart also preserved it in all 96 content-modification runs.

Table 3. E1 outcomes by fault type and policy

Fault	Policy	Runs	Transcript matches	Incorrect commits
Stop only	Full restart	96	96	0
Stop only	Syntax checks	96	96	0
Stop only	Combined	96	96	0
Truncated JSON	Full restart	96	96	0
Truncated JSON	Syntax checks	96	96	0
Truncated JSON	Combined	96	96	0
Text edit	Full restart	96	96	0
Text edit	Syntax checks	96	0	96
Text edit	Combined	96	96	0
Total	Fault runs only	864	768	96

The 96 incorrect commits are repeated observations of the same corruption-policy combination. The key finding is that identical normal-termination status did not imply identical transcript preservation.

Fig. 4 illustrates the distinction between execution completion and transcript preservation. All 96 syntax-based runs under content modification terminated, but none preserved the reference transcript.

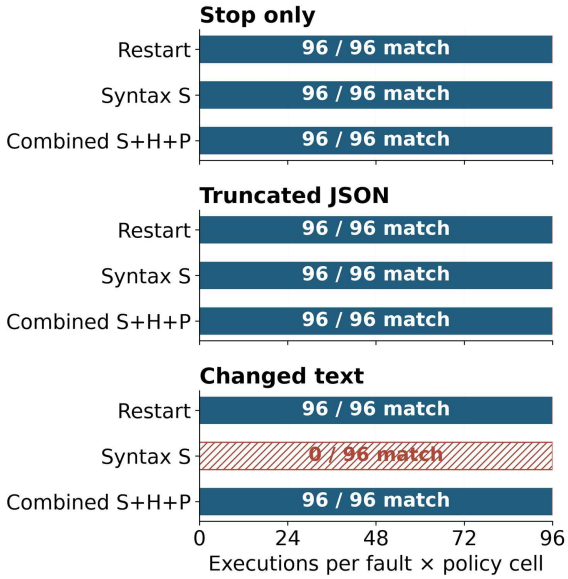


Fig. 4. Transcript preservation by policy in E1's 864 fault-injection runs. Each bar represents 16 recordings × 2 interruption positions × 3 replicates = 96 runs. Hatching denotes incorrect transcript commits; the 48 no-fault controls are excluded. The counts include technical replicates of the same recordings.

2 Recomputation and Latency in Correct Recovery

Table 4 compares combined validation with full restart. In every row, both policies preserved the transcript in all three paired replicates for each of the 16 recordings. After stop-only interruption following the first saved chunk, combined validation processed the remaining 90 seconds, saving 30 seconds of audio recomputation. After the third saved chunk, it processed only the final 30 seconds, saving 90 seconds. The corresponding latency differences were -16.061 and -57.312 seconds.

When the first chunk was truncated or modified at the first interruption position, it also required recomputation. All four chunks, totaling 120 seconds, were therefore processed again. Recomputation savings were zero, and the respective latency differences of -0.001 and +0.023 seconds were close to zero. When the first chunk was corrupted after three chunks had been saved, the second and third chunks remained reusable, saving 60 seconds of audio recomputation.

Table 4. Conditional differences between combined validation and full restart. n: eligible recordings.

Fault	Saved chunks	Recomp. (s)	ΔC (s)	ΔL (s)	n
Stop only	1	90	-30	-16.061	16
Stop only	3	30	-90	-57.312	16
Truncated JSON	1	120	0	-0.001	16
Truncated JSON	3	60	-60	-41.366	16
Text edit	1	120	0	+0.023	16
Text edit	3	60	-60	-41.439	16

ΔL is the equally weighted mean of the recording-level medians of three paired differences. Full restart recomputes 120 seconds after the fault in every E1 condition. Syntax-based reuse achieved similar recomputation savings under stop-only interruption and truncation. Under content modification, however, it produced correctness failures for all 16 recordings at both positions, leaving no eligible observations for a timing comparison conditional on correct recovery.

Fig. 5 adds syntax-based reuse to the comparison, using the aggregation rules applied to combined validation in Table 4. Both reuse policies yield similar savings when recovery is correct. The value of combined validation therefore depends on the conditions under which it permits reuse, as well as the magnitude of the resulting savings.

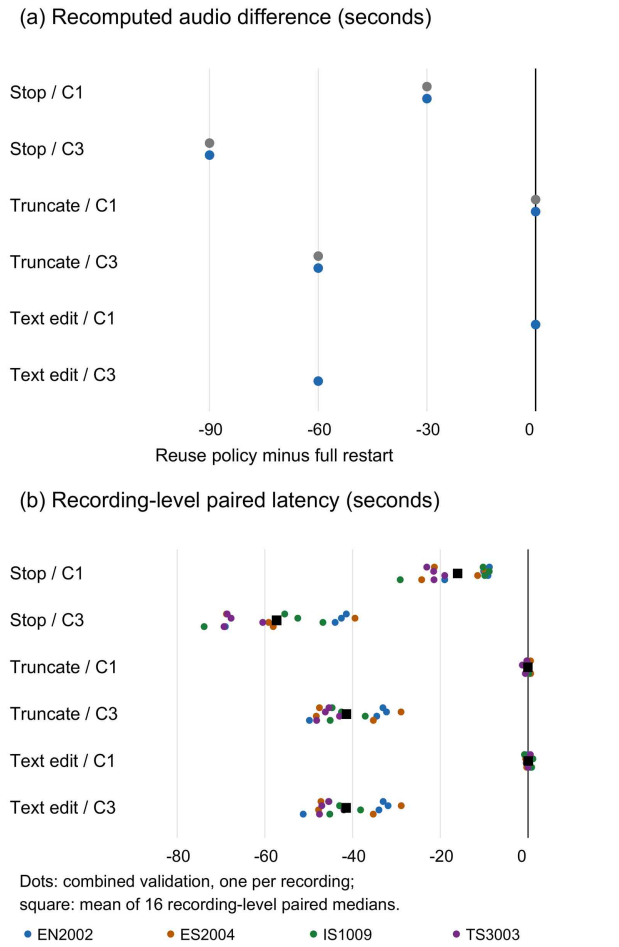


Fig. 5. Differences relative to full restart in (a) recomputed audio duration and (b) recording-level paired latency. In (a), gray denotes syntax-based reuse and blue denotes combined validation; correctness-failure conditions are excluded. In (b), points show recording-level medians of three paired differences for combined validation, and squares show the equally weighted mean across 16 recordings. Colors distinguish the four meeting series.

Table 5 presents a post hoc analysis of the archived E1 records using total elapsed time T, which includes pre-interruption cost. For stop-only interruption at position 3, post-fault latency L was 83.52 seconds for full restart and 26.19 seconds for each reuse policy. The corresponding T values were 145.20, 87.80, and 87.93 seconds. The savings common to both reuse policies reflect reuse of valid artifacts. Additional validation must therefore be assessed in terms of both the conditions under which it preserves correctness and the cost it introduces.

At position 1, corruption of the first chunk produced total elapsed-time differences of +0.066 seconds for truncation and +0.009 seconds for content modification under combined validation. Both were close to zero. Appendix E reports absolute times conditional on correct recovery separately from the total cost of all recorded runs.

Table 5. Post hoc analysis of total elapsed time T for correct recovery

Fault	Pos.	Full restart	Syntax checks	Combined	Runs per policy
Stop only	1	104.01	87.73	87.97	48
Stop only	3	145.20	87.80	87.93	48
Truncated JSON	1	103.82	103.90	103.80	48
Truncated JSON	3	145.36	103.68	103.95	48
Text edit	1	103.77	N/A	103.81	48
Text edit	3	145.24	N/A	103.76	48

All times in Table 5 are in seconds. Each value is the equally weighted mean of the within-recording replicate medians across 16 recordings. Under content modification, all 48 syntax-based runs at each position failed the correctness criterion, so conditional timing is marked N/A. Their cumulative elapsed time of 8,417.51 seconds across 96 runs remains included in the overall cost. Table 4 uses a different aggregation order for paired differences; its estimates should not be equated with differences between columns in Table 5.

3 Storage-Boundary and Development Integration Observations

All 16 E3a cases produced the expected storage states and policy decisions. In three cases, a stale but valid artifact remained alongside its matching hash-validation record. Syntax-based and

output-hash-only policies each accepted all three for incorrect reuse, whereas full restart and combined validation permitted none within this case set. An output-hash match alone can therefore accept an artifact generated for a different task. Appendix C lists the states and identifiers for each storage-boundary case. E3a uses synthetic file contents and execution-condition labels, but terminates the actual file-writing process. These observations are distinct from the two development integration jobs that interrupted actual speech inference.

Using a separate public development meeting, we generated donor artifacts with a different model and a different inference setting, then ran two integrated ASR interruption-and-recovery jobs. The first completed inference for one chunk, received SIGKILL, had an artifact replaced with one from another model, and recovered through combined validation. The second completed inference for three chunks, received SIGKILL, had an artifact replaced with one from another setting, and recovered through full restart. Both recovered outputs matched the reference transcript and structured output. These development cases demonstrate end-to-end interruption and recovery under different conditions.

Table C2 reports implementation and resource-planning evidence from these two runs. Each recomputed 120 seconds of audio, and both used the same development meeting. Neither encountered an unexpected execution failure or timeout. Across 76 service-monitoring samples, no deterioration was detected in services that were healthy at baseline. Pre-interruption inference time, recovery inference time, and host elapsed time were recorded separately.

VI Discussion and Threats to Validity

1 Interpretation of the Research Questions

For RQ1, the results show that parseability does not ensure transcript preservation. Syntax checks accepted altered content when the JSON structure remained valid. For RQ2, reusing valid chunks reduced recomputation and post-fault latency, but offered almost no benefit when all chunks had to be processed again. Recovery policies should therefore be assessed for transcript preservation as well as execution completion, and their costs interpreted in relation to the chunks actually eligible for reuse.

Syntax-based reuse is a minimal-validation control, and its failure under content modification follows from the limited scope of its checks. In particular, a change confined to the text field might also be detected by cross-field consistency checks. E1 consequently provides evidence about basic validation limits and recovery cost. Separating the effects of the hash and execution-condition checks requires the hash-only comparator included in the frozen supplementary experiment. Establishing differences from existing methods likewise requires comparison under equivalent conditions.

2 Scope of Metrics and Reclassification of Earlier Results

Transcript preservation is measured against the same model's uninterrupted output and may therefore retain its original recognition errors. Word error rate (WER) and character error rate

(CER), by contrast, measure recognition quality against a reference transcript. Diarization error rate (DER) additionally requires speaker-time references, defined evaluation intervals, and an explicit policy for overlapping speech. The Korean recognition, diarization, and summarization scores reported in the earlier manuscript have been classified as requiring revalidation of their scoring procedures.

The earlier PoC reported automatic speaker-count agreement of 0/8 and cross-chunk speaker-linking DER of 74.05%. We retain these diagnostic results under their original conditions and distinguish them from E1's recovery evaluation. Speaker-linking performance requires further validation before the system can be assessed for identifying individuals or automatically assigning responsibilities.

3 Generalization and Statistical Interpretation

The evaluation covers the first 120 seconds of 16 recordings from four public English meeting series. Its findings have limited applicability to later portions of long meetings, decoding with cross-chunk context, and Korean construction meetings. We use descriptive statistics to acknowledge dependence within recordings and series and the three technical replicates per condition. Population confidence intervals, significance tests, and tail-latency estimates require independent data and additional measurements.

Prior inspection of the test data and intermediate E1 results also influenced the supplementary design. Validation on an independent corpus should therefore fix the model and experimental conditions before evaluation and disclose any prior access to the data or reference annotations.

The CORRUPTED OUTPUT substitution and half-file truncation are reproducible artificial faults. The findings characterize policy responses to those specific conditions. Inferring operational fault distributions or reliability requires evidence about real-world occurrence rates. Because the supplementary experiment uses the same 16 recordings, it does not replace validation on an independent corpus.

4 Operational and Security Limitations

Absolute latency on a shared, CPU-limited host depends on hardware, load, cache state, and execution order. Service monitoring compared current execution states with previously recorded states as an operational safeguard. Effects on service request latency and availability require separate measurement.

E1 and E3a examine process termination while assuming a trusted controller and trusted validation records. They do not establish protection against simultaneous forgery of artifacts and records, host compromise, or data loss caused by power failure or storage-device damage. Deployment additionally requires procedures for distributing inputs and models, separating privileges, retaining logs, managing keys, and responding to incidents.

5 Reproducibility and Research Ethics

Per-run results, errors, partial files, and frozen specifications are retained in their original form, including all failure cases. The main protocol is identified as KSCI-ARTIFACT-RECOVERY-MAIN-20261026. A separate execution manifest records the relationships among hashes, inputs, code, and schedules. Distribution of these materials requires separate checks of data, code, and model terms and removal of internal connection information and credentials.

Execution evidence is currently retained locally for reproducibility. Public release and independent external reproduction remain pending. The application context and system configuration originate in the earlier conference PoC [17]; this study adds the fault-injection protocol, experiments, and analysis.

6 Total Cost and Conditions for Comparing Existing Methods

An operational cost comparison must account consistently for normal execution, pre-interruption

inference, and post-fault inference, model loading, validation, storage, and control. This paper uses E1 post-fault latency as its primary timing measure and reports a post hoc analysis of total elapsed time, including pre-interruption cost, in Appendix E. Estimating average operational savings also requires measurements of fault frequency.

Comparisons with existing workflows must hold audio, model, settings, chunk dependencies, and resource limits constant. The output-hash-only and combined-validation policies should first be compared to isolate the incremental effect of execution-condition checks. Existing tools can then be configured with equivalent information and reuse units to assess valid-file reuse, content-corruption detection, handling of artifacts from other tasks, and recovery cost. Where correctness is equivalent, comparisons should examine normal-execution overhead and the amount of recomputation. These comparisons, together with independent long-form validation, are needed to extend the present conclusions.

VII Conclusion

This study examined artifact validation and checkpoint recovery within an on-premises meeting intelligence pipeline for construction enterprises. It distinguished artifact syntax, byte integrity, and execution-condition compatibility as criteria for transcript reuse and compared recovery policies through fault injection with the model, inputs, and storage format held constant. All 912 runs terminated, but all 96 runs combining content modification with syntax-based reuse committed incorrect transcripts. Execution completion must therefore be distinguished from transcript preservation. Combined validation preserved transcripts across the tested fault matrix and reduced recomputed audio and post-fault latency when valid checkpoints remained.

Under the specified conditions, content validation detected corruption that preserved file syntax, while cost savings depended on the amount of valid output available for reuse. Policy selection should therefore consider both validation scope and recovery cost. Further evaluation will compare an output-hash-only policy and existing workflows under equivalent conditions, followed by validation on independent long-form data.

References

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, Robust Speech Recognition via Large-Scale Weak Supervision, Proceedings of the 40th International Conference on Machine Learning, PMLR, Vol. 202, pp. 28492–28518, 2023. https://proceedings.mlr.press/v202/radford23a.html

[2] M. Bain, J. Huh, T. Han, and A. Zisserman, WhisperX: Time-Accurate Speech Transcription of Long-Form Audio, INTERSPEECH 2023, pp. 4489-4493, 2023. https://doi.org/10.21437/Interspeech.2023-78

[3] H. Bredin, pyannote.audio 2.1 Speaker Diarization Pipeline: Principle, Benchmark, and Recipe, INTERSPEECH 2023, 2023. https://doi.org/10.21437/Interspeech.2023-105

[4] E. N. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, A Survey of Rollback-Recovery Protocols in Message-Passing Systems, ACM Computing Surveys, Vol. 34, No. 3, pp. 375-408, 2002. https://doi.org/10.1145/568522.568525

[5] M.-C. Hsueh, T. K. Tsai, and R. K. Iyer, Fault Injection Techniques and Tools, Computer, Vol. 30, No. 4, pp. 75-82, 1997. https://doi.org/10.1109/2.585157

[6] Bazel Contributors, Remote Caching, Bazel Documentation. https://bazel.build/remote/caching (accessed 2026-10-29)

[7] P. Di Tommaso, M. Chatzou, E. W. Floden, P. P. Barja, E. Palumbo, and C. Notredame, Nextflow Enables Reproducible Computational Workflows, Nature Biotechnology, Vol. 35, pp. 316-319, 2017. https://doi.org/10.1038/nbt.3820

[8] Nextflow Contributors, Caching and Resuming, Seqera Documentation. https://docs.seqera.io/nextflow/cache-and-resume (accessed 2026-10-29)

[9] Iterative, Running Pipelines, DVC Documentation. https://doc.dvc.org/user-guide/pipelines/running-pipelines (accessed 2026-10-29)

[10] Seqera, Getting Started with Data Lineage, Nextflow Documentation, experimental feature introduced in version 25.04. https://docs.seqera.io/nextflow/tutorials/data-lineage (accessed 2026-10-29)

[11] Snakemake Contributors, Between Workflow Caching, Snakemake 8.20.4 Documentation. https://snakemake.readthedocs.io/en/v8.20.4/executing/caching.html (accessed 2026-10-29)

[12] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswany, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications, OSDI 14, pp. 433-448, 2014. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/pillai

[13] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, Zero Trust Architecture, NIST SP 800-207, 2020. https://doi.org/10.6028/NIST.SP.800-207

[14] AMI Consortium, AMI Corpus. https://groups.inf.ed.ac.uk/ami/corpus/ (accessed 2026-10-29)

[15] pyannote Contributors, AMI Diarization Setup, a fork of the BUT Speech@FIT setup, revision 67c2d539286e89f68952d5dcf83912bd9f01dfae. https://github.com/pyannote/AMI-diarization-setup/tree/67c2d539286e89f68952d5dcf83912bd9f01dfae

[16] S. H. Hurlbert, Pseudoreplication and the Design of Ecological Field Experiments, Ecological Monographs, Vol. 54, No. 2, pp. 187-211, 1984. https://doi.org/10.2307/1942661

[17] J. Kim, J. Yun, J. Lee, and K. Kim, A Secure On-Premises Meeting Intelligence Pipeline Using Edge AI for Construction Enterprises, Proceedings of the 74th Summer Conference of the Korea Society of Computer and Information, Jeju, Republic of Korea, July 2026.

Appendix A Evaluation Scope and Further Validation

Tables 3 and 4 derive from the completed E1 experiment; Table C2 reports two completed integration jobs using a separate public development meeting. Each protocol is summarized separately using its own denominator. The evidence reported here is limited to results completed and reviewed when the manuscript was finalized on October 29, 2026.

Follow-up work addresses an output-hash-only comparator, artifacts generated from different inputs, models, or settings, and validation on an independent corpus. Results from the ongoing supplementary main experiment are maintained separately from the completed E1 experiment and development cases.

Quality assessment against official reference annotations is a separate task. Equivalence between scores obtained with the existing custom normalization and the original distribution's scoring procedure has yet to be established. Speaker-attribution evaluation also requires speaker-time references. The present paper evaluates transcript preservation relative to an uninterrupted run with the same input and model.

Aggregation proceeds from paired differences for individual replicates to the median for each recording, then to the mean of the 16 recording-level medians. If, for example, a recording has three differences of -10, -12, and -30 seconds, its representative difference is -12 seconds; these are hypothetical values. Taking the difference between separately calculated policy medians may give a different result. Absolute-time means and paired differences in Appendix E follow their respective aggregation procedures.

Appendix B Variation Across Recordings in the Existing Results

Table B1 presents a post hoc descriptive analysis of recording-level paired differences using the estimator and eligibility criteria in Table 4. Ranges indicate observed minima and maxima. Each series mean gives equal weight to its four recordings.

Table B1. Distribution of latency differences for combined validation and means by series

Fault	Min-max (s)	EN2002	ES2004	IS1009	TS3003
Stop only, pos. 1	-29.131 ~ -8.801	-11.757	-16.722	-14.513	-21.251
Stop only, pos. 3	-73.841 ~ -39.440	-49.238	-56.353	-57.140	-66.518
Truncated JSON, pos. 1	-1.287 ~ 0.613	0.037	0.416	0.030	-0.487
Truncated JSON, pos. 3	-49.834 ~ -28.943	-37.416	-40.007	-42.350	-45.693
Text edit, pos. 1	-0.798 ~ 1.117	0.158	-0.465	0.394	0.007
Text edit, pos. 3	-51.253 ~ -28.901	-37.542	-39.787	-42.928	-45.498

Conditions with both small positive and small negative differences indicate variation around zero relative to full restart. The three technical replicates characterize variation under the specified execution conditions.

Appendix C Execution Specification and Storage-Boundary Observations

E1 uses 16 kHz mono PCM audio from AMI Mix-Headset, 30-second chunks, OpenAI Whisper large-v3-turbo, two CPU threads, English, temperature 0, and beam size 1. Both fp16 and conditioning on preceding chunk transcripts are disabled. Containers are limited to the equivalent of 2 CPUs and 8 GiB of memory, with no GPU or external network access. The accompanying execution_manifest.json retains complete identifiers for the image, model, code, inputs, and schedule. Checking Python, PyTorch, and Whisper versions against the frozen image, and independently reproducing the experiment, remain follow-up tasks.

E3a examines eight storage positions from each of two initial states: an empty state and a state containing a valid artifact-record pair generated under different input conditions. The file-writing process is terminated at the first seven positions; the final position is an uninterrupted control. This yields 16 cases. Temporary files are ineligible for reuse, and Table C1 describes artifacts and validation records at their final paths. New files contain synthetic data labeled with the current execution conditions; old files contain synthetic data associated with a different input.

Table C1. Expected and observed states at storage boundaries. Entries show artifact / validation-record states.

Case IDs	Interruption / control point	Empty start	Old pair
00 / 08	Before opening the artifact temporary file	None / none	Old / old
01 / 09	After partially writing the artifact temporary file	None / none	Old / old
02 / 10	After syncing the artifact temporary file, before replacement	None / none	Old / old
03 / 11	After artifact replacement, before writing its record	New / none	New / old
04 / 12	After partially writing the validation-record temporary file	New / none	New / old
05 / 13	After syncing the record temporary file, before replacement	New / none	New / old
06 / 14	After record replacement, before the completion marker	New / new	New / new
07 / 15	Uninterrupted control	New / new	New / new

Case numbers identify archived records e3a-plan-00 through e3a-plan-15. All 16 cases matched the expected states in Table C1. In cases 08, 09, and 10, the old artifact remained alongside its matching hash-validation record, so syntax and output-hash checks alone could not prevent incorrect reuse. Combined validation rejected the pair because its input conditions differed from those expected for the current task. These observations

demonstrate the file-selection rules at the specified storage boundaries.

Both development ASR integration jobs used the same public development meeting. In the different-model case, tiny.en generated the donor artifact and large-v3-turbo was the recovery target. In the different-setting case, an artifact generated by large-v3-turbo with beam size 2 was substituted into a task requiring beam size 1. Interruption followed inference for the first and third chunks, respectively. The corresponding recovery policies were combined validation and full restart. Table C2 reports the observed interruption-and-recovery integration and execution costs for these two development cases.

Table C2. Cost breakdown of the development ASR integration jobs

Development case	Pre-stop inference (s)	Recovery inference (s)	Total inference (s)	Host (s)
Different model; pos. 1; combined	25.811	116.741	142.552	155.433
Different setting; pos. 3; full restart	85.552	117.222	202.775	215.426

Appendix D Relationship Between the PoC Materials and This Experiment

Fig. 6 shows the PoC equipment used to illustrate the enterprise meeting-transcription context: a Raspberry Pi 5 board and a development setup connecting a GIGABYTE DGX Spark to an audio interface and capture devices.

The project interim report identifies speech capture in secure environments, transcription, speaker differentiation, and meeting summarization as development objectives. It documents checks of a meeting-record list and interfaces displaying participant-specific transcripts and summaries. These materials illustrate how transcript artifacts are viewed and summarized downstream. Because reuse of an incorrect intermediate artifact can propagate errors to later stages, the present evaluation focuses on transcript-file reuse decisions and recovery cost.

The quantitative evaluation uses public AMI recordings. The PoC photographs document the application environment linking speech-capture devices with computing equipment.



(a) (b)

Fig. 6. PoC equipment: (a) Raspberry Pi 5 board; (b) GIGABYTE DGX Spark and connected peripherals.

Source: PoC equipment photographs supplied by the authors and the Collaboration Project Interim Report (August 17, 2026).

Appendix E Post Hoc Analysis of Total Elapsed Time

This post hoc analysis reexamined the 912 archived result.json files after E1 was completed. Each condition contains 48 runs, with three replicates for each of 16 recordings. For runs returning correct results, we calculate the replicate median for each recording and then average these medians with equal weights across all 16 recordings. Values are in seconds, with policies ordered as full restart, syntax-based reuse, and combined validation. Values outside parentheses denote post-fault latency L; those in parentheses denote total elapsed time T, including pre-interruption inference.

Stop only, position 1: 83.63 (104.01), 67.39 (87.73), 67.64 (87.97).

Stop only, position 3: 83.52 (145.20), 26.19 (87.80), 26.19 (87.93).

JSON truncation, position 1: 83.55 (103.82), 83.59 (103.90), 83.49 (103.80).

JSON truncation, position 3: 83.49 (145.36), 42.06 (103.68), 42.12 (103.95).

Content modification, position 1: 83.48 (103.77), N/A, 83.49 (103.81).

Content modification, position 3: 83.53 (145.24), N/A, 42.09 (103.76).

Under content modification, all 48 syntax-based runs at each position returned incorrect transcripts, so timing conditional on correct recovery is marked N/A. These failures remain included in the overall outcomes and cost totals. The cumulative elapsed time of the 96 runs was 8417.51 seconds.

Summing per-run T across all 48 no-fault controls and 864 fault-injection runs gives 95043.46 seconds (26.40 hours). This cumulative duration includes both pre-interruption inference and recovery. SHA-256 identifiers for the raw records, condition-level and recording-level aggregates, and analysis code are retained with the separate reproducibility materials.

All E1 policies write validation records to provide consistent instrumentation. Isolating the cost of record writing and the full validation mechanism requires a no-fault comparison that varies only validation functionality while retaining the same storage procedure.

Appendix F Transcription and Summarization Service Interface

Fig. 7 shows the PoC transcription and summarization interface during development with lecture audio. Users can inspect the chronological transcript on the left and the summary on the right. The screenshot illustrates how stored transcripts provide input to downstream summarization.

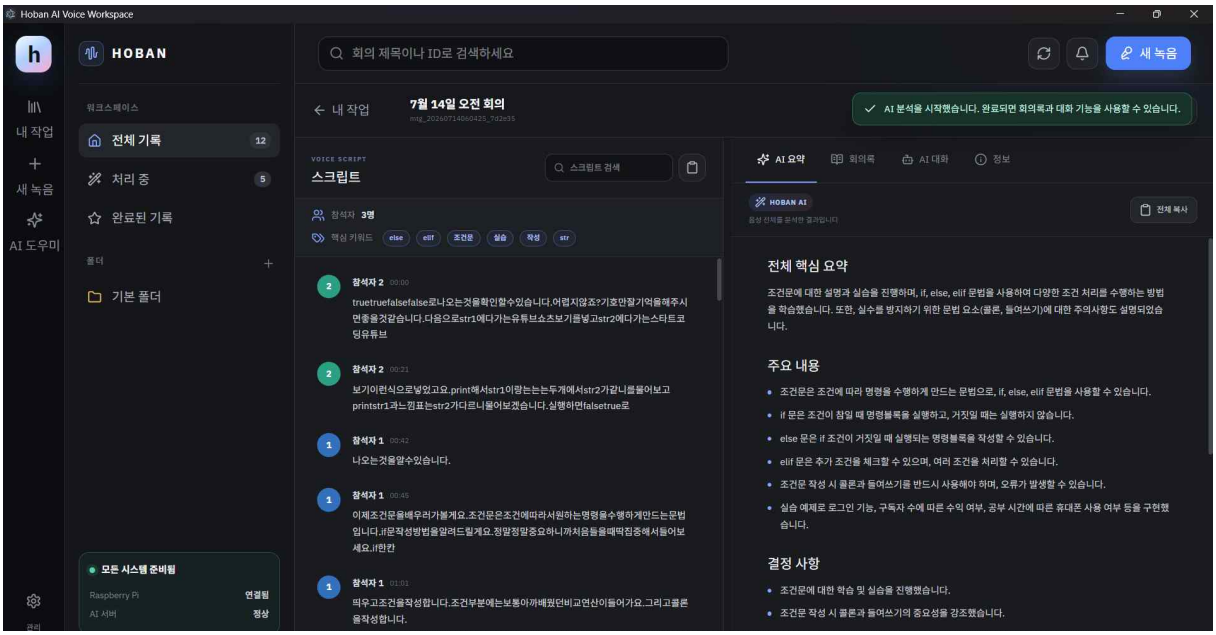


Fig. 7. Development screenshot of the PoC transcription and summarization service using lecture audio.

Source: Service screenshot supplied by the authors (development material from July 2026).

Authors



Jaehwan Kim is a Ph.D. candidate at aSSIST (Seoul School of Integrated Sciences and Technologies), Seoul, Republic of Korea. He contributed to the overall system design, pipeline implementation, empirical measurements, and manuscript preparation. His research interests include on-premises meeting intelligence and edge AI.



Jaeseong Yun is an AI engineer at XAIKOREA Inc., Seongnam, Republic of Korea. He contributed to speech AI processing, vLLM serving, and concurrent processing for meeting summarization.



Jongseok Lee leads the Infrastructure Team at RAD CNS Co., Ltd., Republic of Korea. He contributed to the two-node GPU infrastructure and service orchestration.



Kyungmin Kim is a principal researcher in the Technical Support Team at Security Platform, Republic of Korea. He contributed to the security architecture, AES-256-GCM protection, and Vault KEK integration.